

MAMBA-NIKE Reference Implementation x86 Self-Evaluation Report

MAMBA-NIKE Submission Team

June 30, 2026

Document Information

Algorithm	MAMBA-NIKE
Implementation track	Reference
Target architecture	x86-64
Implementation directory	API_PKC/Implementations/Reference_Implementation/
Parameter authority	parameters.json
Parameter SHA-256	a34156a3dcc6c80328e5bf8814d7c1aa4764dd1ac8e257c455000e124194ffdd
Evaluation code	ci/reference_self_evaluation.py
Machine-readable evidence	ci/reference_self_evaluation_data.json
Tabular evidence	ci/reference_self_evaluation_data.csv

1 Scope

This report evaluates only the submitted Reference implementation. It does not combine evidence from the other implementation track. The evaluated profiles are MAMBA-NIKE-128, MAMBA-NIKE-192, MAMBA-NIKE-256, MAMBA-NIKE-384, and MAMBA-NIKE-512. The evaluation covers dependencies, the frozen parameter record, implementation-boundary enforcement, canonical known-answer tests, protocol agreement observed by the benchmark harness, and x86 cycle measurements.

The scheme operates in

$$R_q = \mathbb{Z}_q[X]/(X^n + 1), \quad q = 8192 = 2^{13},$$

and uses SHAKE256 for transcript-bound shared-secret derivation. No Cortex-M4 or Additional Implementation material is part of this report.

2 Dependencies

- Python 3.9 or newer; only the Python standard library is used.
- GNU Make and Bash.
- A GCC-compatible C99 compiler.
- An x86-64 host. No AVX2 instruction-set support is required.
- No third-party cryptographic library is required.

The evaluator itself has no network dependency and does not install packages. The optimized evaluator stops with a failure when AVX2 is unavailable instead of silently reporting portable fallback results as optimized results.

3 Implementation Boundary

The evaluated implementation is portable coefficient-domain C. Its ring multiplication path is `poly_convolution()` to `toom4_mul()`. The submitted Reference tree contains no AVX2 intrinsics, AVX2 assembly, AVX2 preprocessor selection, or AVX2 compiler options. Compatibility entry points named `poly_ntt()` and `poly_invntt()` are no-ops and do not implement an NTT.

- The source-boundary scan rejects `immintrin.h`, `_mm256_*`, `__m256`, `__AVX2__`, `-mavx2`, `-march=native`, and `poly_mul_small`.
- Every profile must retain both `poly_convolution()` and `toom4_mul()` in its multiplication path.
- KAT binaries are built in temporary directories so the submitted implementation tree is not polluted with build products.

4 Self-Evaluation Procedure

The implementation-specific runner performs the following ordered checks:

1. verify required tools and processor capabilities;
2. verify the immutable `parameters.json` SHA-256 value;
3. scan the selected implementation tree for the documented implementation boundary;
4. copy each profile to a temporary directory;
5. build and run its KAT executable;
6. compare the generated KAT file byte for byte with `API_PKC/Test_Vectors/`;
7. run the submitted 1,000-iteration RDTSC benchmark;
8. verify zero observed agreement failures;
9. write JSON and CSV evidence; and
10. verify that `parameters.json` was unchanged.

```
# Run from the repository root.
python3 ci/reference_self_evaluation.py
# Structural and KAT checks without the 1,000-run benchmark:
python3 ci/reference_self_evaluation.py --skip-benchmark
```

5 Known-Answer Test Results

All five generated KAT files matched the canonical Reference test-vector files byte for byte. The digest prefix below is included for convenient evidence identification; the complete digest is stored in the JSON evidence file.

Table 1: Reference KAT reproduction results.

Profile	File bytes	Result	SHA-256 prefix
MAMBA-NIKE-128	213180	PASS	85ea2c7e0762ca3e...
MAMBA-NIKE-192	223420	PASS	965a521ebbc5fd6d...
MAMBA-NIKE-256	223420	PASS	cb420026a0962881...
MAMBA-NIKE-384	464700	PASS	eb125000e598fbca...
MAMBA-NIKE-512	465340	PASS	a340e7bfd2002220...

6 x86 Performance and Agreement Data

The submitted benchmark harness uses RDTSC and reports the arithmetic mean over 1,000 timed executions after warm-up. KeyGen, responder Pass1, and initiator Derive are measured separately. The same 1,000 executions also check shared secret agreement.

Table 2: Reference results (kCycles; 1,000 executions).

Profile	n	PK B	M_1 B	KeyGen	Pass1	Derive	Agree / Fail
MAMBA-NIKE-128	1024	1184	1568	1056	2065	1005	1000 / 0
MAMBA-NIKE-192	1024	1312	1568	1058	2066	1007	1000 / 0
MAMBA-NIKE-256	1024	1312	1568	1071	2083	1011	1000 / 0
MAMBA-NIKE-384	2048	2848	3360	2140	4219	2085	1000 / 0
MAMBA-NIKE-512	2048	2848	3360	2139	4212	2069	1000 / 0

All recorded benchmark exchanges agreed and the observed failure count was zero. This empirical result is not presented as a cryptographic upper bound on the decryption or reconciliation failure probability.

7 Data Format

The JSON evidence records environment metadata, dependency status, the implementation boundary, the parameter hash before and after evaluation, full KAT digests, timing data, agreement counts, and provenance notes. The CSV file contains one normalized benchmark row per submitted profile. Both files are plain UTF-8 and can be consumed without a project-specific parser.

8 Security and Correctness Limitations

This implementation self-evaluation verifies reproducibility and consistency of the submitted code. It does not replace independent cryptanalysis, formal verification, constant-time leakage testing, or a certified numerical failure probability analysis. The parameter manifest and every profile-specific `params.h` remain frozen and are not rewritten by the evaluator.

9 Conclusion

The Reference implementation passed the implementation-specific source boundary checks and reproduced all five canonical KAT files. The associated 1,000-run x86 benchmark dataset recorded zero shared-secret disagreements. The evaluation code, JSON evidence, and CSV evidence are separated from the other implementation track so that the submitted Reference and Optimized implementations can be assessed independently.